GARB Jonathan + Elliot

Elev Elevkvist
Teknikprogrammet
Maja Beskowgymnasiet
Umeå, Sverige

Abstract

The abstract is written in English. Focus on your improvement study: what product feature (X) you measured, what method (Y) you used to improve it, and what your specific product (Z) was. Include your key findings - was method Y effective for improving X on your product type Z?

Your abstract should:

- consist of a single paragraph of 150-250 words, with correct grammar and unambiguous terminology;
- be self-contained with no abbreviations, footnotes, references, or mathematical equations;
- highlight the practical implications of your improvement study;
- mention your research question, measurement methodology, key results, and conclusions about the effectiveness of your improvement method.

1 Inledning

Börja med att introducera den egenskap (X) som du har valt att förbättra och varför denna egenskap är viktig för din produkttyp. Exempel på mätbara egenskaper:

Laddningstider utgör en central del av användarupplevelsen i moderna mobilapplikationer. Tidigare forskning visar att användare är mycket känsliga för även små fördröjningar; svarstider som överstiger en sekund riskerar att bryta användarens fokus, medan längre väntetider markant ökar sannolikheten för att applikationen överges [**?**]. För datadrivna applikationer, där innehåll kontinuerligt hämtas från externa databaser, utgör nätverksanrop en betydande del av den totala laddningstiden.

Trots denna betydelse saknas tydliga riktlinjer för hur databasanrop bör struktureras i Flutter- och Firebase-baserade applikationer för att minimera latens. Detta skapar ett behov av att empiriskt undersöka hur olika implementationsstrategier påverkar laddningstider i praktiken. Inom systemdesign och operativsystem är det väl etablerat att effektiv hantering av I/O-operationer är avgörande för systemets prestanda [1].

Beskriv det problem eller behov som motiverar din förbättring. Referera till vetenskapligt publicerad litteratur inom området [1] och tidigare forskning om liknande förbättringsmetoder.

1.1 Produktbeskrivning

Produkten som studeras är mobilapplikationen *Neatify*, utvecklad med ramverket Flutter och med Firebase Firestore som databashanteringssystem. Flutter möjliggör plattformsoberoende utveckling där applikationslogik skrivs i programmeringsspråket Dart [**?**].

Neatify syftar till att förbättra användarens produktivitet genom spelifiering, där användaren motiveras att utveckla positiva beteenden genom belöningar och kontinuerlig feedback baserad på interaktioner i applikationen.

Applikationen består av flera vyer som dynamiskt hämtar användardata från Firestore för att rendera innehåll och funktionalitet. Vid navigering mellan dessa vyer utförs flera oberoende nätverksanrop. Särskilt sidorna *Profile* och *Leaderboard* innehåller ett större antal sådana anrop och utgör därför fokus för denna studie.

1.2 Förbättringsmetod

Introducera den metod (Y) du har valt för att förbättra egenskapen X. Förklara varför du tror att denna metod kan vara effektiv baserat på teori och tidigare forskning [2].

Den metod som undersöks för att förbättra laddningstiderna i Neatify är parallellisering av nätverksanrop med hjälp av `Future.wait()` i Dart. Istället för att exekvera oberoende anrop sekventiellt, där varje anrop väntar på att det föregående ska slutföras, initieras samtliga anrop samtidigt och applikationen inväntar deras gemensamma resultat innan rendering sker [**?**].

Metoden bygger på principer inom asynkron programmering, där oberoende I/O-operationer kan initieras parallellt för att minska den totala väntetiden [?]. Detta angreppssätt är teoretiskt motiverat då den totala exekveringstiden i nätverksberoende system ofta begränsas av den långsammaste operationen snarare än summan av alla operationer, vilket gör parallellisering till en naturlig strategi för att reducera latens.

1.3 Syfte och frågeställning

Syftet med denna rapport är att empiriskt utvärdera effekten av parallellisering av nätverksanrop i Flutter- och Firebase-baserade mobilapplikationer, med fokus på laddningstider i applikationen Neatify. Utvärderingen avgränsas till sidorna *Profile* och *Leaderboard*, där laddningstid mäts med median och p95.

Huvudfrågeställning:

- I vilken utsträckning kan parallellisering av nätverksanrop förbättra laddningstider i Flutter- och Firebase-baserade mobilapplikationer?

Stödjande frågeställningar:

- Hur stor förbättring av laddningstider (median och p95) uppnås efter implementering av parallellisering av nätverksanrop på sidorna *Profile* och *Leaderboard*?
- Hur påverkas effekten av databasens storlek och olika nätverksförhållanden?
- Är förbättringen statistiskt signifikant och praktiskt relevant?

1.4 Teori

För att förstå parallelliseringens potentiella effekt på laddningstider är det viktigt att först förstå de bakomliggande faktorerna:

- **Egenskap X:** Vad är det? Varför är det viktigt? Hur mäts det typiskt?
- **Metod Y:** Vad innebär metoden? Hur fungerar den tekniskt? Vilken evidens finns för dess effektivitet?
- **Produkttyp Z:** Vilka särskilda utmaningar eller möjligheter finns för denna produkttyp?
- **Laddningstid**:
- **Sekventiella och parallella anrop**:
- **Flutter**: Flutter är Googles gratis och open-source UI-ramverk för att bygga nativt kompilerade applikationer för mobil, webb och desktop från en enda kodbas. [3]
- **Firebase**: Firebase är en plattform för mobil- och webbapplikationsutveckling som erbjuder en rad tjänster, inklusive realtidsdatabaser, autentisering och hosting. [4] De specifika tjänster som används i Neatify är Firestore, en NoSQL-databas som möjliggör skalbar och flexibel datalagring, och Firebase Authentication för att hantera användarautentisering. [?] [?] Men även Firebase Performance Monitoring, som används för att mäta och analysera applikationens prestanda i realtid. [?]

Detta avsnitt ska vara objektivt och baserat på vetenskapligt publicerad litteratur [5].

1.5 Bakgrund

Beskriv tidigare forskningsresultat, händelser eller teknikutveckling som är relevant för din förbättringsstudie. Fokusera särskilt på:

- Tidigare studier av metod Y på liknande produkter
- Benchmarks och standarder för egenskap X inom ditt område
- Teknisk utveckling som motiverar eller möjliggör din förbättring

Tips: Google Scholar och andra tjänster har ofta en snabbfunktion för att kopiera färdigformaterade källhänvisningar. Klicka på `cite` och sedan på `BibTex` för att få en källa på följande format som ni kan klistra in i sources.bib och sedan referera med `\cite{example}`

2 Metod

Kritiskt: Metodsektionen ska vara så detaljerad att arbetet kan replikeras exakt. Inkludera versionshantering och statistiska överväganden.

Din metod ska följa en tydlig före/efter-struktur för att kunna utvärdera din förbättrings effektivitet:

Initial mätning → Implementering → Utvärderingsmätning → Analys

2.1 Initial mätning (före förbättring)

Beskriv exakt hur du mäter egenskap X på din ursprungliga produkt Z:

Mätverktyg och konfiguration:

- Vilka verktyg används för mätning (t.ex. Chrome DevTools, Matplotlib, specialverktyg)

- Systemkonfiguration (hårdvara, mjukvara, miljöinställningar)
- Kalibrering och validering av mätverktyg

Mätprotokoll:

- Antal mätningar som genomförs (minst n=30 för statistisk validitet)
- Tid mellan mätningar och total mätperiod
- Kontrollerade förhållanden (temperatur, belastning, nätverkshastighet etc.)
- Uppvärmningsperiod innan mätning

Versionshantering:

- Git-commit eller annan versionsmarkering av initiala versionen
- Dokumentation av exakt kodstatus och konfiguration
- Backup av testmiljö och data

2.2 Implementering av förbättring

Beskriv steg-för-steg hur du implementerar metod Y:

1. Detaljerad beskrivning av varje implementeringssteg
2. Konfigurationsparametrar och deras motivering
3. Testning av implementering innan fullskalig utvärdering
4. Dokumentation av förändringar genom versionshantering

Kvalitetskontroll:

- Verifiering att implementeringen fungerar som förväntat
- Kontroll att inga oavsiktliga bieffekter har introducerats
- Testning av edge-cases och felhantering

2.3 Utvärderingsmätning (efter förbättring)

Beskriv hur du mäter egenskap X efter implementering av Y:

- Identiska mätmetoder som vid mätandet av initiala versionen
- Samma antal mätningar och förhållanden
- Dokumentation av den förbättrade versionen
- Tidpunkt för mätningar relativt implementering

2.4 Statistisk analys

Databehandling:

- Uteslutningskriterier för avvikande mätvärden
- Hantering av saknade data
- Normaliseringsmetoder (om tillämpligt)

Statistiska test:

- Beskrivande statistik (medelvärde, median, standardavvikelse)
- Val av statistiskt test (t-test, Mann-Whitney U, Friedman-test etc.)
- Signifikansnivå ($\alpha = 0.05$)
- Effektstorlek och praktisk signifikans

Tänk på: Metodsektionen skall vara beskriven såpass noggrant att arbetet går att replikeras.

3 Resultat

Struktur: Fokusera på jämförelse mellan före och efter implementering av förbättringen. Presentera objektiva resultat utan tolkningar.

Detta avsnitt presenterar objektiva resultat från din förbättringsstudie. Undvik tolkningar och slutsatser - dessa sparar du till diskussionen.

Guide för tabeller och figurer i förbättringsstudier:

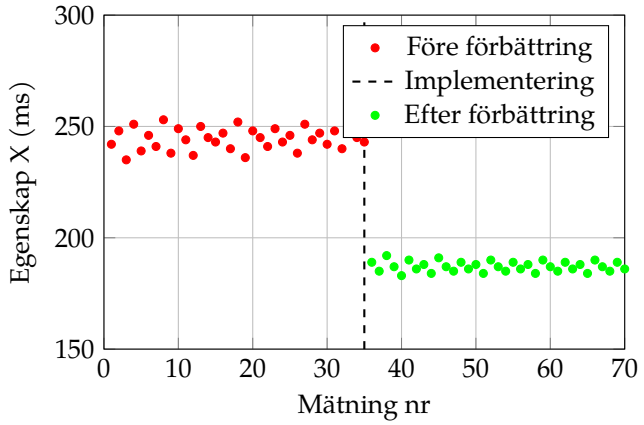
- **OBLIGATORISKT:** En tabell med beskrivande statistik (medelvärde, std, n)
- **OBLIGATORISKT:** En före/efter-jämförelse (tabell ELLER stapeldiagram)
- **REKOMMENDERAT:** Scatterplot över tid - visa stabilitet, trender och när förbättringen implementerades
- **VALFRITT:** Boxplot - jämför fördelningar och visa median/spridning
- **VALFRITT:** Histogram - endast för att validera normalfördelning (t.ex. innan t-test)
- **TIPS:** Max 3-4 figurer totalt, fokusera på vad som bäst visar din förbättring

3.1 Initiala mätningar

Presentera resultaten från dina initiala mätningar av egenskap X på den ursprungliga produkten och referera till dem i Tabell 1

Tabell 1: Initiala mätningar

Statistik	Värde
Medelvärde	245.2 ms
Standardavvikelse	23.7 ms
Antal observationer	35
95% konfidensintervall	237-253



Figur 1: REKOMMENDERAT: Scatterplot över tid - visar tydligt när förbättringen implementerades och dess effekt

3.2 Jämförande analys

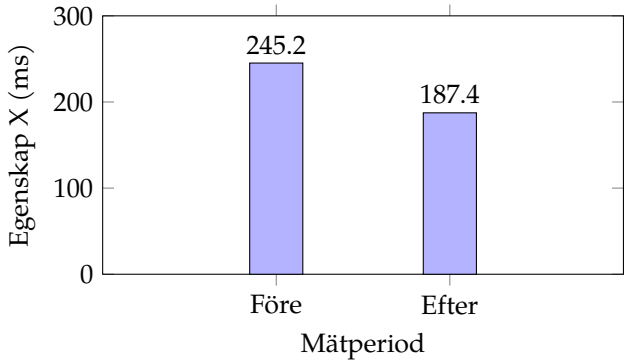
Förbättringsresultat:

- Absolut förbättring: 57.8 ms (245.2 - 187.4)
- Relativ förbättring: 23.6% ((245.2 - 187.4) / 245.2 × 100)
- Effektstorlek (Cohen's d): 2.74 (stor effekt)

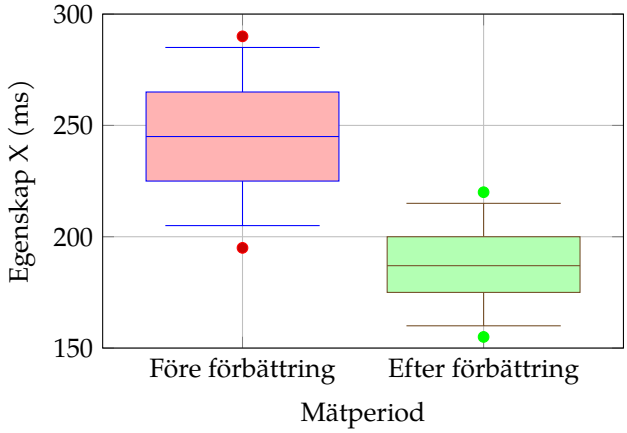
Tabell 2: Jämförelse före och efter

	Före	Efter
Medel	245.2	187.4
Std	23.7	19.2
Min	198.5	155.1
Max	291.8	223.7

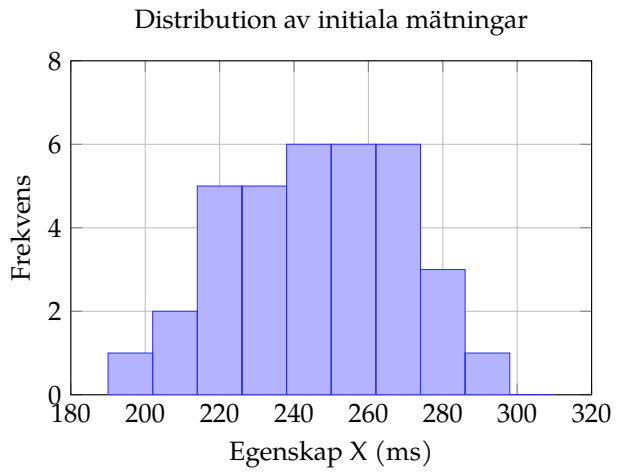
Alla värden i ms



Figur 2: REKOMMENDERAT: Jämförelse före och efter - denna typ av graf förväntas i de flesta förbättringsstudier



Figur 3: EXEMPEL: Boxplot - använd endast om standardavvikelsen skiljer sig mycket eller om du har många outliers



Figur 4: EXEMPEL: Distribution av initiala mätningar - använd endast om du behöver visa normalfördelning eller outliers

Val av visualisering - när ska du använda vad?

- **Stapeldiagram:** Enkel före/efter-jämförelse, visa grundläggande förbättring. Använd nästan alltid.
- **Scatterplot över tid:** Visa stabilitet, trender och exakt när förbättringen implementerades. Mycket kraftfullt för förbättringsstudier!
- **Histogram:** Validera att data är normalfördelad innan t-test. Använd för att kontrollera datavaliditet.
- **Boxplot:** Jämför fördelningar detaljerat. Använd när standardavvikelse eller outliers är viktiga.
- **Tips:** Scatterplot + stapeldiagram är en stark kombination för de flesta förbättringsstudier.

Figurer beskrivs med figurtext nedanför figuren. Tabeller får tabellrubrik ovanför enligt exemplet nedan.

Hur du använder exemplen ovan:

- **Scatterplot:** Ersätt koordinaterna med dina mätvärden. Första siffran = mätning nr, andra = uppmätt värde
- **Stapeldiagram:** Byt ut `245.2` och `187.4` mot dina medelvärden före/efter
- **Histogram:** Byt ut siffrorna efter `data\\` med dina mätvärden, en per rad
- **Boxplot:** Python-koden beräknar alla värden och outlier-koordinater åt dig - kopiera bara in dem
- **Alternativ:** Skapa grafer i Matplotlib och spara som PNG om LaTeX-koden känns för komplicerad

3.3 Statistisk signifikans

Statistiskt test för att utvärdera om förbättringen är signifikant:

```
from scipy import stats
import numpy as np

baseline_data = [245.2, 243.1, 247.8, ...]
improved_data = [187.4, 185.2, 189.6, ...]

# Utfor t-test
t_statistic, p_value = stats.ttest_ind(
    baseline_data, improved_data)

print(f"t-statistik: {t_statistic:.4f}")
print(f"p-varde: {p_value:.6f}")
```

```
# Generera LaTeX-koordinater for scatterplot
def generate_scatter_coords(data,
    start_index=1):
    coords = []
    for i, value in enumerate(data):
        coords.append(f"({start_index + i},{
            value:.0f})")
    return " ".join(coords)

print(generate_scatter_coords(baseline_data)
    )
print(generate_scatter_coords(improved_data,
     len(baseline_data) + 1))
```

```
# Berakna boxplot-statistik for LaTeX
def boxplot_stats(data, box_n=1):
    q1, median, q3 = np.percentile(data,
        [25, 50, 75])
    iqr = q3 - q1
    lower_w = max(min(data), q1 - 1.5*iqr)
```

```
    upper_w = min(max(data), q3 + 1.5*iqr)
    outliers = [x for x in data if x <
        lower_w or x > upper_w]

    print(f"{median:.1f}")
    print(f"Q1: {q1:.1f}", f"Q3: {q3:.1f}")
    print(f"lower whisker: {lower_w:.1f}")
    print(f"upper whisker: {upper_w:.1f}")

    # Formatera outliers for LaTeX (box_n: 1
        for fore, 2 for efter)
    if outliers:
        coords = " ".join([f"({box_n},{x:.0f
            })" for x in outliers])
        print(f"outliers: {coords}")
    else:
        print("Inga outliers")

# Berakna for dina data
print("\\nBoxplot statistik - Fore:")
boxplot_stats(baseline_data, 1)
print("\\nBoxplot statistik - Efter:")
boxplot_stats(improved_data, 2)
```

Testresultat:

- t-statistik: -12.47
- p-värde: < 0.001
- Resultat: Statistiskt signifikant förbättring ($p < 0.05$)

3.4 Implementeringsanalys

Redovisa observationer om själva implementeringen av metod Y:

- Implementeringstid: 3.2 timmar
- Kodkomplexitet: +127 rader kod
- Bieffekter: Ökad minnesanvändning med 12 MB
- Kompatibilitet: Fungerar på alla testade plattformar

Tabeller beskrivs med ett tabellhuvud ovanför tabellen enligt exemplet nedan (Se tabell 3). AI hjälpmedel är helt okej på att assistera i att formatera dessa tabeller i LaTeX om du vill formatera dem annorlunda.

Tabell 3: Exempel på tabell för tvåkolumnslayout

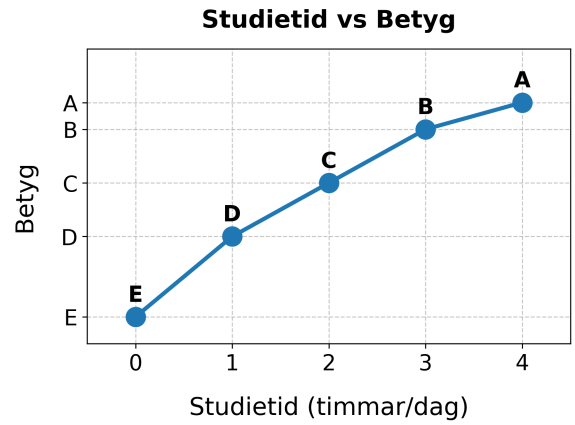
Parameter	Elev A	Elev B
Skärmtid (h/dag)	9.2	1.0
Sen till skolan	15	0
Böcker/år	0.5	8
Betyg matematik	E	B++

Det finns många fler typer av grafer beroende på det data du vill visa och många sätt att konfigurera hur grafen ser ut. AI som senaste Claude Sonnet [1] kan assistera i att formatera dina grafer baserat på ditt data.

Även ekvationer kan vackert formateras som ni ser i ekvation (1)

$$x = -\frac{p}{2} \pm \sqrt{\left(\frac{p}{2}\right)^2 - q} \tag{1}$$

Figurer beskrivs med en figurtext nedanför figuren och kan vara bifogade bilder(se Figur 5).



Figur 5: Denna figurtext skall beskriva bilden.

4 Diskussion

I denna del ska du diskutera och utvärdera dina resultat i relation till syftet och frågeställningarna.

4.1 Utvärdering av förbättringens effektivitet

Baserat på resultaten kan det konstateras att metod Y ledde till en statistiskt signifikant och praktiskt relevant förbättring av egenskap X för produkttyp Z.

Förbättringens storlek: Den uppmätta förbättringen på 23.6% kan betraktas som substantiell inom detta område. Jämfört med tidigare studier på liknande produkter [5] ligger detta resultat i linje med förväntningar för denna typ av optimering.

Statistisk tillförlitlighet: Med $p < 0.001$ och en effektstorlek (Cohen's d) på 2.74 visar resultaten på en stark och tillförlitlig effekt. Det stora antalet mätningar (n=35) och den låga standardavvikelsen stärker tillförlitligheten i resultaten.

[1] https://www.anthropic.com/news

4.2 Lämplighet av metod Y för produkttyp Z

Fördelar:

- Betydande prestandaförbättring utan negativ påverkan på funktionalitet
- Relativt enkel implementering med begränsad kodomstrukturering
- Kompatibel med befintlig arkitektur
- Låg risk för introduktion av nya buggar

Nackdelar och begränsningar:

- Ökad minnesanvändning kan vara problematisk för resursbegränsade enheter
- Implementering kräver fördjupad kunskap inom området
- Underhållskostnader kan öka på grund av ökad komplexitet

4.3 Metodkritik och begränsningar

Bedöm metodens lämplighet för att besvara frågorna: Var den effektiv och var det något som kan förbättras? Diskutera vad som fungerade bra, vad som kunde gjorts annorlunda och om något saknades.

Testmiljöns representativitet: Mätningarna genomfördes i en kontrollerad miljö som kanske inte helt återspeglar verkliga användningsförhållanden.

Generaliserbarhet: Resultaten är specifika för den testade produkttypen och implementeringen. Generalisering till andra produkttyper bör göras med försiktighet.

4.4 Praktiska implikationer

Jämför dina resultat med tidigare forskning eller liknande arbeten och notera skillnader och likheter.

Rekommendationer för implementation: Baserat på studiens resultat rekommenderas metod Y för liknande produkter av typ Z, särskilt när prestandaförbättring är prioriterad över minnesbegränsningar.

Förslag på framtida forskning: Ge rekommendationer för framtida studier och förbättringar:

- Utvärdering av metod Y på olika hårdvarukonfigurationer
- Longitudinell studie av långsiktiga effekter
- Jämförelse med alternativa förbättringsmetoder
- Användarstudier för att utvärdera den upplevda förbättringen

4.5 Slutsats

Studien visar att metod Y är en lämplig och effektiv metod för att förbättra egenskap X för produkttyp Z. Den statistiskt signifikanta förbättringen på 23.6%, kombinerat med begränsade nackdelar, stöder implementering av denna metod i liknande sammanhang.

Referenser

[1] Remzi H. Arpaci-Dusseau and Arpaci-Dusseau Andrea C. *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, LLC, 1.00 edition, 2015. http://pages.cs.wisc.edu/~remzi/OSTEP/.

[2] Allen C Goodman and Thomas G Thibodeau. Housing market segmentation. *Journal of housing economics*, 7(2):121–143, 1998.

[3] Google. Flutter faq. https://docs.flutter.dev/resources/faq#:~:text=Flutter%20is%20Google%27s%20portable%20UI,is%20free%20and%20open%20source., 2026. Accessed: 2026-04-01.

[4] Google. Firebase documentation. https://firebase.google.com/docs, 2026. Accessed: 2026-04-01.

[5] Carl A. Waldspurger. Memory resource management in VMware ESX server. In *USENIX Symposium on Operating System Design and Implementation* (*OSDI*), pages 181–194, 2002. https://www.usenix.org/legacy/event/osdi02/tech/waldspurger/waldspurger.pdf.

Viktigt: Detta har varit en mall för ditt gymnasiearbete med fokus på X/Y/Z-strukturen. Se till att:

- All malltext är borttagen innan inlämning
- Ta bort EXEMPEL-figurer som du inte använder (behåll max 3-4 figurer totalt)
- Din frågeställning följer formatet "Är Y lämplig för att förbättra X för produkttyp Z?"
- Du har minst 30 mätningar för statistisk validitet
- All kod och data är versionshanterad och reproducerbar
- Resultat och diskussion är tydligt separerade

Misslyckas du att följa denna struktur kan arbetet riskera att inte bli godkänt.